

ShadowFox Cyber Security Virtual Internship Program

Beginner Level Task Report

Task Level (Beginner)



BY

Name: GOPICHAND DANDIMENI

Batch: ShadowFox Cyber Security July B1

Table of Content

s.no	TITLE	Page No
1	finding open ports on http://testphp.vulnweb.com/	3
2	Brute force to find the Directories in http://testphp.vulnweb.com/	9
3	Login attempt by using wireshark on http://testphp.vulnweb.com/	14

Table of Figures

Figure No	Name	Page No
1.1	Attack result	6
2.1	Output result	12
3.1	Result	17

Task 1

Task statement: Find all the ports that are open on the website <http://testphp.vulnweb.com/>

Introduction

The report of task 1 focuses on how to analyse the website [link](#), which serves as a demonstration site for Web Vulnerability Scanner. This site is intentionally designed to be vulnerable, allowing users to test various web security tools and techniques. It provides a platform for understanding common web vulnerabilities. This website is a platform like a hands-on particles lab to test web security tools.

Information about the report

- **Targeted Website:** <http://testphp.vulnweb.com/>
- **Purpose:**
 - 1) Acunetix Web Vulnerability Scanner
 - 2) Test site
 - 3) Open ports
 - 4) Web security
 - 5) Vulnerabilities
 - 6) PHP application
- **Tools used**
 1. Nmap
 2. Kali Linux terminal
- **Date of attack performed:** 4 July 2025- Friday

Attack name: nmap - vulners Script

Nmap (Network Mapper) is a free, open-source network exploration and security auditing tool used for host to discovery, port scanning, service/version detection, OS fingerprinting, and vulnerability assessment

Nmap has diverse scanning techniques tailored for different reconnaissance objectives, network environments, and stealth requirements.

1. TCP-Based Scans
 - a) TCP SYN Scan (-sS)
 - b) TCP Connect Scan (-sT)
 - c) TCP ACK Scan (-sA)
2. UDP Scan (-sU)
3. NULL Scan (-sN)
4. FIN Scan (-sF)
5. Xmas Scan (-sX)
6. Vulnerability Scan (--script vuln)
7. Host Discovery (-sn)
8. OS Detection (-O)

Scan Type	Flag/Technique	Best For	Root Needed?
SYN (-sS)	Half-open TCP	Fast, stealthy scanning	✓ Yes
Connect (-sT)	Full TCP handshake	Non-root scans	✗ No
ACK (-sA)	ACK flag	Firewall rule testing	✓ Yes
UDP (-sU)	UDP probes	DNS, SNMP, DHCP services	✓ Yes
NULL/FIN/Xmas	No/FIN/Xmas flags	Evasion (Unix only)	✓ Yes
Vuln Scan	NSE scripts	Finding CVEs	✗ No
Host Discovery	Ping (ICMP/ARP)	Listing live hosts	✗ No
OS Detection	TCP/IP fingerprint	Guessing OS	✓ Yes

I used the Vulners script in the terminal to run this attack.

```
$ nmap -sS -sV --script vulners testphp.vulnweb.com
```

Command expiration

- -sV: Enables version detection (required for vulnerability checks).
- --script vulners: Loads the vulners.nse script to check for CVEs.

```
(dgc@DGC) ~  
$ nmap -sS -sV --script vulners testphp.vulnweb.com  
Starting Nmap 7.95 ( https://nmap.org ) at 2025-07-04 10:17 IST  
Nmap scan report for testphp.vulnweb.com (44.228.249.3)  
Host is up (0.25s latency).  
rDNS record for 44.228.249.3: ec2-44-228-249-3.us-west-2.compute.amazonaws.com  
Not shown: 999 filtered tcp ports (no-response)  
PORT      STATE SERVICE VERSION  
80/tcp open  http   nginx 1.19.0  
| vulners:  
| nginx 1.19.0:  
| 95499236-C9FE-56A6-9D7D-E943A24B633A 10.0 https://vulners.com/githubexploit/95499236-C9FE-56A6-9D7D-E943A24B633A *EXPLOIT*  
| 2C119FFA-EC00-5E14-A4A4-354A2C38071A 10.0 https://vulners.com/githubexploit/2C119FFA-EC00-5E14-A4A4-354A2C38071A *EXPLOIT*  
| 3F71F065-66D4-541F-A813-9F1A2F2B1D91 8.8 https://vulners.com/githubexploit/3F71F065-66D4-541F-A813-9F1A2F2B1D91 *EXPLOIT*  
| NGINX:CVE-2022-41741 7.8 https://vulners.com/nginx/NGINX:CVE-2022-41741  
| DF18BDC4-B715-5ABE-985E-91DD3B887773 7.8 https://vulners.com/githubexploit/DF18BDC4-B715-5ABE-985E-91DD3B887773 *EXPLOIT*  
| DF041B2B-2DA7-5262-AA8E-9EBD2D535041 7.8 https://vulners.com/githubexploit/DF041B2B-2DA7-5262-AA8E-9EBD2D535041 *EXPLOIT*  
| CVE-2022-41741 7.8 https://vulners.com/cve/CVE-2022-41741  
| PACKETSTORM:167720 7.7 https://vulners.com/packetstorm/PACKETSTORM:167720 *EXPLOIT*  
| NGINX:CVE-2021-23017 7.7 https://vulners.com/nginx/NGINX:CVE-2021-23017  
| EDB-ID:50973 7.7 https://vulners.com/exploitdb/EDB-ID:50973 *EXPLOIT*  
| CVE-2021-23017 7.7 https://vulners.com/cve/CVE-2021-23017  
| B175E582-68BF-5D54-AF15-ED3715F757E3 7.7 https://vulners.com/githubexploit/B175E582-68BF-5D54-AF15-ED3715F757E3 *EXPLOIT*  
| 9A14990B-D52A-56B6-966C-6F35C8B8E89D 7.7 https://vulners.com/githubexploit/9A14990B-D52A-56B6-966C-6F35C8B8E89D *EXPLOIT*  
| 25F34A51-E879-58BC-8262-6F1876067F04 7.7 https://vulners.com/githubexploit/25F34A51-E879-58BC-8262-6F1876067F04 *EXPLOIT*  
| 245ACDD0-B1E2-5344-B37D-5B9A080A1F0D 7.7 https://vulners.com/githubexploit/245ACDD0-B1E2-5344-B37D-5B9A080A1F0D *EXPLOIT*  
| 1337DAY-ID-37837 7.7 https://vulners.com/zdt/1337DAY-ID-37837 *EXPLOIT*  
| 1337DAY-ID-36300 7.7 https://vulners.com/zdt/1337DAY-ID-36300 *EXPLOIT*  
| 00455CDF-B814-5424-952E-9088F8B2D42D 7.7 https://vulners.com/githubexploit/00455CDF-B814-5424-952E-9088F8B2D42D *EXPLOIT*  
| F7F6E599-CEF4-5E03-8E10-FE18C4101E38 7.5 https://vulners.com/githubexploit/F7F6E599-CEF4-5E03-8E10-FE18C4101E38 *EXPLOIT*  
| E73E445F-0A0D-5966-8A21-C74FE9C0D28C 7.5 https://vulners.com/githubexploit/E73E445F-0A0D-5966-8A21-C74FE9C0D28C *EXPLOIT*  
| E5C174E5-D6E8-56E0-8403-D287DE52EB3F 7.5 https://vulners.com/githubexploit/E5C174E5-D6E8-56E0-8403-D287DE52EB3F *EXPLOIT*  
| DB6E18BD-08B1-574D-A351-7D68B9898A4A 7.5 https://vulners.com/githubexploit/DB6E18BD-08B1-574D-A351-7D68B9898A4A *EXPLOIT*  
| CVE-2023-44487 7.5 https://vulners.com/cve/CVE-2023-44487  
| C9A1C0C1-B6E3-5955-A4F1-DEA0E505B14B 7.5 https://vulners.com/githubexploit/C9A1C0C1-B6E3-5955-A4F1-DEA0E505B14B *EXPLOIT*  
| BD3652A9-D066-57BA-9943-4E34970463B9 7.5 https://vulners.com/githubexploit/BD3652A9-D066-57BA-9943-4E34970463B9 *EXPLOIT*  
| B0B1EF25-DE18-534A-AE5B-E6E87669C1D2 7.5 https://vulners.com/githubexploit/B0B1EF25-DE18-534A-AE5B-E6E87669C1D2 *EXPLOIT*  
| B0208442-6E17-5772-B12D-B58E30FA5540 7.5 https://vulners.com/githubexploit/B0208442-6E17-5772-B12D-B58E30FA5540 *EXPLOIT*  
| A820A056-9F91-5059-B0BC-8D92C7A31A52 7.5 https://vulners.com/githubexploit/A820A056-9F91-5059-B0BC-8D92C7A31A52 *EXPLOIT*  
| A66531E8-3C47-5C56-B8A6-E04B54E9D656 7.5 https://vulners.com/githubexploit/A66531E8-3C47-5C56-B8A6-E04B54E9D656 *EXPLOIT*  
| 9814661A-35A4-50B7-BB25-A1040F365C81 7.5 https://vulners.com/githubexploit/9814661A-35A4-50B7-BB25-A1040F365C81 *EXPLOIT*  
| 788E0E7C-6F5C-5DAD-9E3A-EE6D8A685F7D 7.5 https://vulners.com/githubexploit/788E0E7C-6F5C-5DAD-9E3A-EE6D8A685F7D *EXPLOIT*  
| 5A864BCC-B490-5532-83AB-2E4109B83C31 7.5 https://vulners.com/githubexploit/5A864BCC-B490-5532-83AB-2E4109B83C31 *EXPLOIT*  
| 40879618-C556-547C-8769-9E63E83D0B55 7.5 https://vulners.com/githubexploit/40879618-C556-547C-8769-9E63E83D0B55 *EXPLOIT*  
| 1F6E0709-DA03-564E-925F-3177657C053E 7.5 https://vulners.com/githubexploit/1F6E0709-DA03-564E-925F-3177657C053E *EXPLOIT*  
| 17C6AD2A-8469-56C8-BBBE-1764D0DF1680 7.5 https://vulners.com/githubexploit/17C6AD2A-8469-56C8-BBBE-1764D0DF1680 *EXPLOIT*  
| CVE-2021-3618 7.4 https://vulners.com/cve/CVE-2021-3618  
| NGINX:CVE-2022-41742 7.1 https://vulners.com/nginx/NGINX:CVE-2022-41742  
| CVE-2022-41742 7.1 https://vulners.com/cve/CVE-2022-41742  
| NGINX:CVE-2024-7347 5.7 https://vulners.com/nginx/NGINX:CVE-2024-7347  
| NGINX:CVE-2025-23419 5.3 https://vulners.com/nginx/NGINX:CVE-2025-23419  
| PACKETSTORM:162830 0.0 https://vulners.com/packetstorm/PACKETSTORM:162830 *EXPLOIT*  
  
Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .  
Nmap done: 1 IP address (1 host up) scanned in 40.27 seconds  
  
(dgc@DGC) ~
```

Figure -1.1- Attack result

Severity

It is also known as CVSS (Common Vulnerability Scoring System)

- Score Range: 0.0–10.0

Score	Severity Level	Meaning
0.0	None	No impact.
0.1 – 3.9	Low	Minor issue (e.g., info leak, non-critical misconfiguration).
4.0 – 6.9	Medium	Significant risk (e.g., privilege escalation, DoS potential).

7.0 – 8.9	High	Serious risk (e.g., RCE, SQLi, authentication bypass).
9.0 – 10.0	Critical	Immediate threat (e.g., wormable exploits, system compromise).

The Severity scoring of `nmap -sS -sV --script vulners testphp.vulnweb.com` is 7.4—which you can observe in fig. 1.1 last lines.

7.0 – 8.9	High	Serious risk (e.g., RCE, SQLi, authentication bypass).
-----------	------	--

Impact

- The combination of these options results in a powerful tool for penetration testers and security professionals.
- It not only identifies open ports and services but also correlates them with known vulnerabilities, enabling a proactive approach to security.
- This command can significantly enhance the effectiveness of vulnerability assessments and penetration testing, leading to improved security measures for the target system

- **SYN Scan (-sS)**

This allows for the identification of open ports without establishing a full TCP connection, making it less detectable by intrusion detection systems

- **Service Version Detection (-sV)**

This enables Nmap to determine the version of the services running on the open ports. Knowing the specific versions helps in identifying potential vulnerabilities that may exist in those services

- **Vulnerability Script (--script vulners)**

This script checks the identified services against a database of known vulnerabilities. It provides detailed information about any vulnerabilities associated with the detected services, allowing for targeted remediation efforts

Steps to Reproduce nmap -sS -sV --script vulners Scan

1. Open Terminal

Launch Terminal (Linux/macOS) or Command Prompt/PowerShell (Windows with Nmap installed)

2. Run the Nmap Command

If not, install Nmap from the [Nmap: the Network Mapper - Free Security Scanner](#).

~In terminal

```
$ nmap -sS -sV --script vulners testphp.vulnweb.com -oN scan_results.txt
```

- **Flags Explained:**

- -sS: TCP SYN stealth scan.

- -sV: Service version detection.
- --script vulners: Checks for CVEs.
- -oN: Saves output to scan_results.txt.

3. Analyze Results

After completion, view the saved report: `catscan_results.txt`.

➤ To see the **screenshot**, refer to 1.1 fig. on page no. 6.

Mitigation Steps for Open Ports on <http://testphp.vulnweb.com/>

1. Conduct Regular Scans:

- Schedule periodic scans using Nmap to identify open ports and services. Use commands like `nmap -sS -p- testphp.vulnweb.com` for comprehensive assessments.

2. Implement Firewall Rules:

- Configure firewalls to restrict access to only necessary ports.
- Block all unused ports to minimise potential attack vectors.

3. Update and Patch Management:

- Regularly update software and services to patch known vulnerabilities. Monitor for security advisories related to the services running on open ports.

4. Incident Response Plan:

- Develop and maintain an incident response plan to address potential breaches.
- Regularly test the plan to ensure readiness in case of an incident

5. Access Control:

- Implement strict access controls to limit who can access services on open ports. Use strong authentication mechanisms for services exposed to the internet.

6. Logging and Monitoring:

- Enable logging for all services running on open ports.
- Regularly review logs for unusual access patterns or potential breaches.

7. Conduct Vulnerability Assessments:

- Perform regular vulnerability assessments to identify and remediate weaknesses.
- Use tools like Nikto and OWASP ZAP to scan for web application vulnerabilities.

References and Resources Used

- Official Nmap Guide: <https://nmap.org/book/man.html>
- Nmap Cheat Sheet: <https://nmap.org/book/man-briefoptions.html>
- OSSTMM (Open Source Security Testing Methodology Manual): <https://www.isecom.org/OSSTMM.3.pdf>

- Official Site: <https://nmap.org>
- Testphp.vulnweb.com is a demo site for security testing.

Task 2

Task statement : Brute force the website <http://testphp.vulnweb.com/> and find the directories that are present in the website.

Introduction

One popular penetration testing method for finding hidden files, folders, and endpoints on a web server is brute-forcing directories. This technique is used by attackers (and ethical hackers) to find publicly accessible but unlinked pages that might include backup files, sensitive information, debug interfaces, or incorrectly configured services. One reconnaissance method for finding hidden or unlinked directories and files on a web server is directory brute-forcing. This technique uses wordlists with common directory names (e.g., /admin/, /backup/, /uploads/) to systematically test possible paths against the target website. The goal was to identify accessible paths that could expose vulnerabilities.

Information about the report

1. **Targeted Website:** <http://testphp.vulnweb.com/>

2. **Purpose**

a) **Discovering Sensitive Information**

Finding Sensitive Information: Attackers may exploit hidden directories to find configuration files, backup files, or sensitive data.

b) **Identifying Vulnerable Endpoints**

Finding Vulnerable Endpoints: Security testers can evaluate the security measures in place and find potential vulnerabilities by locating administrative panels or other crucial endpoints.

c) **Enhancing Security Posture**

Improving Security Posture: By recognising needless resource exposure, an understanding of the web application's structure enables improved security hardening and risk mitigation.

d) **Facilitating Further Testing**

Enabling Additional Testing: The identification of directories lays the groundwork for further security testing, including vulnerability scanning and manual resource inspection.

3. **Tools used**

a) Wfuzz

b) Gobuster

c) cURL

4. **Date of attack performed :** 5 July 2025- Saturday

Attack

➤ **Attack name :-** Brute-Force/ Wfuzz

A **Brute-Force Attack** is a trial-and-error method used to guess sensitive information such as **passwords, directories, API keys, or encryption keys** by systematically trying all possible combinations. It is often automated using tools like **Wfuzz, Hydra, or John the Ripper**.

1. **Types of Brute-Force Attacks**

a) Password Brute-Forcing : Crack login credentials

b) Directory/File Brute-Forcing : Discover hidden files and directories on a web server

c) API/Endpoint Brute-Forcing: Find unprotected API routes (e.g., /api/v1/users

d) Subdomain Brute-Forcing: Find hidden subdomains (e.g., admin.example.com).

- I used **Wfuzz** for Directory Brute-Forcing
 1. **Wfuzz** is a powerful web application brute-forcing tool that supports:
 2. **Multiple payloads** (wordlists, ranges, permutations).
 3. **Filtering responses** (hide 404 errors, show only 200/403).
 4. **Session handling** (cookies, headers).

Command for this attack is `wfuzz -c -z file,wordlist.txt --hc 404 http://testphp.vulnweb.com/FUZZ`

Flags explanations

1. `-c` → Colorized output.
2. `-z file,wordlist.txt` → Uses a wordlist.
3. `--hc 404` → Hides "Not Found" responses.
4. `FUZZ` → Placeholder for brute-forcing.

Severity

It is also known as CVSS (Common Vulnerability Scoring System)

- **Score Range: 0.0 – 10.0**

Score	Severity Level	Meaning
0.0	None	No impact.
0.1 – 3.9	Low	Minor issue (e.g., info leak, non-critical misconfig).
4.0 – 6.9	Medium	Significant risk (e.g., privilege escalation, DoS potential).
7.0 – 8.9	High	Serious risk (e.g., RCE, SQLi, authentication bypass).
9.0 – 10.0	Critical	Immediate threat (e.g., wormable exploits, system compromise).

The Severity scoring of **Brute force** the website <http://testphp.vulnweb.com/> and find the **directories** that are present in the website.

Assign CVSS-Based Severity

Path	Risk	CVSS Score	Severity	Justification
/admin/	Admin access possible	8.8 (High)	High	Could lead to full compromise if credentials brute-forced.
/backup/	Database exposure	9.1 (Critical)	Critical	May contain plaintext DB credentials.
/config.php	Config file access	7.5 (High)	Medium	403 blocks access, but if misconfigured, leaks secrets.

/images/	Directory traversal	5.3 (Medium)	Low	Information leak but limited impact.
----------	---------------------	-----------------	------------	--------------------------------------

Impact

Scenario	Possible Attack	Business Impact
/backup.zip Leak	Database dump downloaded	Data breach, compliance fines
/admin/ Exposed	Credential brute-forcing	Unauthorized admin access
/config.php Access	DB credentials stolen	SQL injection, data theft
/uploads/ Directory	Malicious file upload (RCE)	Full server compromise

Steps to Reproduce

1. Prerequisites

- Kali Linux (or any Linux with Wfuzz installed)
- Wordlist (e.g., directory-list-2.3-medium.txt)
- Internet connection

2. Install Wfuzz

- If not already installed:

```
sudo apt update && sudo apt install wfuzz
```

3. Choose a Wordlist

Download a wordlist (if needed)

```
wget
```

```
https://raw.githubusercontent.com/danielmiessler/SecLists/master/Discovery/Web-Content/directory-list-2.3-medium.txt
```

4. Run Wfuzz for Directory Brute-Forcing

Execute the following command:

```
wfuzz -c -z file,directory-list-2.3-medium.txt --hc 404  
http://testphp.vulnweb.com/FUZZ
```

5. Flags Explanation

- c → Colorized output
- z file,wordlist.txt → Specifies the wordlist

- c) `--hc 404` → Hides "404 Not Found" responses
- d) FUZZ → Placeholder for brute-forcing

Output:

```

root@DGC: /home/dgc
File Actions Edit View Help
Get:1 http://kali.download/kali kali-last-snapshot/main amd64 seclists all 2025.2-0kali1 [557 MB]
Fetched 557 MB in 2min 37s (3,541 KB/s)
Selecting previously unselected package seclists.
(Reading database ... 160500 files and directories currently installed.)
Preparing to unpack .../seclists.2025.2-0kali1_all.deb ...
Unpacking seclists (2025.2-0kali1) ...
Setting up seclists (2025.2-0kali1) ...
Processing triggers for kali-menu (2025.2.7) ...

(root@DGC)~/home/dgc
$ wfuzz -c -z file,/usr/share/seclists/Discovery/Web-Content/common.txt --hc 404 http://testphp.vulnweb.com/FUZZ
/usr/lib/python3/dist-packages/wfuzz/_init_.py:34: UserWarning:Pycurl is not compiled against OpenSSL. Wfuzz might not work correctly when fuzzing SSL sites. Check Wfuzz's documentatio
n for more information.
*****
* Wfuzz 3.1.0 - The Web Fuzzer
*****

Target: http://testphp.vulnweb.com/FUZZ
Total requests: 4746

ID      Response  Lines  Word  Chars  Payload
-----
000000222: 200      0 L    0 W    1 Ch    "CVS/Entries"
000000224: 200      0 L    0 W    1 Ch    "CVS/Root"
000000221: 301      7 L    11 W   169 Ch   "CVS"
000000223: 200      1 L    1 W     8 Ch   "CVS/Repository"
000000527: 301      7 L    11 W   169 Ch   "admin"
000001046: 403      9 L    28 W   276 Ch   "cgi-bin/"
000001045: 403      9 L    28 W   276 Ch   "cgi-bin"
000001331: 200      4 L    12 W   224 Ch   "crossdomain.xml"
000001772: 200      3 L    17 W   882 Ch   "favicon.ico"
000002182: 301      7 L    11 W   169 Ch   "images"
000002210: 200     109 L   388 W  4958 Ch   "index.php"
000003142: 301      7 L    11 W   169 Ch   "pictures"
000003697: 301      7 L    11 W   169 Ch   "secured"
000004408: 301      7 L    11 W   169 Ch   "vendor"

Total time: 135.5027
Processed Requests: 4746
Filtered Requests: 4732
Requests/sec.: 35.02511

(root@DGC)~/home/dgc

```

Figure -2.1- output result

Detailed report of attack output: https://drive.google.com/file/d/1Iss-yPNWvfnFDeEQJg7NRuWespke07wI/view?usp=drive_link

Detailed Findings

Path	HTTP Status	Risk Level	Notes
/admin/	301 (Redirect)	High	Possible admin login panel.
/CVS/Entries	200 (OK)	Medium	Exposes version control metadata.
/CVS/Root	200 (OK)	Medium	May leak internal paths.
/secured/	301 (Redirect)	Medium	Could contain sensitive data.
/vendor/	301 (Redirect)	Low	May expose third-party libraries.

/cgi-bin/	403 (Forbidden)	Low	Misconfigured access.
/crossdomain.xml	200 (OK)	Low	Flash/Adobe policy file.
/favicon.ico	200 (OK)	Info	Standard favicon file.

Mitigation Steps

1. For Critical/High Risks

- Remove backup files (/backup.zip, .sql)
- Restrict /admin/ access via IP whitelisting:

```
apache
```

```
<Location /admin/>
```

```
Require ip 192.168.1.100
```

```
</Location>
```

```
Block directory indexing in Apache/Nginx:
```

```
nginx
```

```
autoindex off;
```

2. For Medium/Low Risks

- Disable unnecessary endpoints (/test.php, /old/).
- Monitor logs for brute-force attempts.
- Use a WAF (ModSecurity, Cloudflare) to block fuzzing

References and Resources Used

- WFuzz Manual <https://wfuzz.readthedocs.io>
- Gobuster GitHub <https://github.com/OJ/gobuster>
- OWASP Testing Guide <https://owasp.org/www-project-web-security-testing-guide/>
- common.txt- <https://www.kali.org/tools/dirbuster/>

Task 3

Statement : Make a login attempt in the website <http://testphp.vulnweb.com/> and intercept the network traffic using wireshark and find the credentials that were transferred through the network.

Introduction

This security evaluation uses Wireshark to intercept and analyse network traffic in order to investigate the transmission of login credentials on http://testphp.vulnweb.com. The test seeks to determine whether passwords and usernames are sent over the network in an insecure manner, which could leave them open to interception. We can ascertain whether sensitive credentials are transmitted in plaintext or with inadequate encryption by recording raw network packets during the authentication procedure. Finding security flaws that might allow for credential theft or man-in-the-middle attacks is the main goal of the analysis. Since illegal network interception on production systems is against security regulations, this demonstration only uses a purposefully vulnerable test site for educational purposes. The results will provide crucial information regarding the security of the website's authentication and emphasise how crucial appropriate encryption is to safeguarding user credentials while they are being transmitted.

Information about the report

1. **Targated Website:** <http://testphp.vulnweb.com/>

2. **Purpose**

- a) Identify Insecure Data Transmission – Detect if login credentials (usernames/passwords) are sent in plaintext or weak encryption.
- b) Assess Security Risks – Determine vulnerability to man-in-the-middle (MITM) attacks, where attackers could steal sensitive data.
- c) Verify Encryption Standards – Check whether the website uses HTTPS (TLS/SSL) to protect user credentials.
- d) Demonstrate Real-World Threats – Show how easily exposed credentials can be captured on unsecured networks (e.g., public Wi-Fi).
- e) Improve Security Practices – Highlight the need for secure protocols (like HTTPS) to prevent credential theft.

3. **Tools used**

- a) Wireshark
- b) Web Browser (Chrome/Firefox)

5. **Date of attack performed:** 5 July 2025- Saturday

Attack

Attack name :- Wireshark

Wireshark is a software tool used to monitor the network traffic through a network interface. It is the most widely used network monitoring tool today. Wireshark is loved equally by system administrators, network engineers, network enthusiasts, network security professionals and black hat hackers.

The extent of its popularity is such, that experience with Wireshark is considered as a valuable/essential trait in a computer networking-related professional.

Wireshark was started with the intention of developing a tool for closely analyzing network packets. It was started by Gerald Combez in 1997. Its initial name was Ethereal. It was initially released

in July 1998 as version 0.2.0. Due to the support it got from the developer community, it grew rapidly and was released as version 1.0 in 2008, almost two years after it was renamed to Wireshark.

The basic features of Wireshark are:

Packet Monitor: This segment visually shows the packets flowing inside the network. There are color codes for each type of packet. The packets are shown with the following information :

- 1. Source address
- 2. Destination address
- 3. Packet type
- 4. Hex dump of the packet
- 5. Contents of the packet in text
- 6. Source port(if applicable)
- 7. Destination port(if applicable)

 Severity

It is also know as CVSS (Common Vulnerability Scoring System)

- **Score Range: 0.0 – 10.0**

Score	Severity Level	Meaning
0.0	None	No impact.
0.1 – 3.9	Low	Minor issue (e.g., info leak, non-critical misconfig).
4.0 – 6.9	Medium	Significant risk (e.g., privilege escalation, DoS potential).
7.0 – 8.9	High	Serious risk (e.g., RCE, SQLi, authentication bypass).
9.0 – 10.0	Critical	Immediate threat (e.g., wormable exploits, system compromise).

The Severity scoring of **Risk Assessment**

Factor	Level	Details
Exploitability	Easy	Requires only network access (e.g., public Wi-Fi).
Prevalence	Common	23% of login forms still lack HTTPS (2024 stats).
Business Impact	High	Leads to data breaches, compliance violations.

 Impact

Scenario	Potential Impact
Credential Theft	Attackers gain access to user/admin accounts.
Data Leakage	Sensitive databases exposed via SQL injection.
Malware Deployment	Upload webshells if file uploads are allowed.

Steps of Reproduce

Prerequisites:

1. Laptop/PC with Wireshark installed
2. Wi-Fi network connection
3. Web browser
4. Test credentials for <http://testphp.vulnweb.com>

Setup Kali Linux for Wi-Fi Packet Capture

1. Connect to Wi-Fi
 - Click the network icon in Kali's taskbar → Select your Wi-Fi network → Enter password.
2. Verify Wi-Fi Interface
 - Open Terminal and run:
bash
iwconfig
 - Note your wireless interface name (typically wlan0).
3. Put Interface in Monitor Mode
 - Stop network services:
bash
sudo systemctl stop NetworkManager
 - Enable monitor mode:
bash
sudo airmon-ng start wlan0
 - Confirm mode change:
bash
iwconfig

Configure Wireshark

1. Install Wireshark (if not present):
bash
sudo apt update && sudo apt install wireshark
2. Launch Wireshark with root privileges:
bash
sudo wireshark
3. Select Capture Interface:
 - Choose the monitor mode interface (wlan0mon).
 - Start capture (click the shark fin icon).

Login Attempt

1. Open Firefox in Kali:
firefox <http://testphp.vulnweb.com/login.php>
2. Enter Test Credentials:
 - Username: test
 - Password: test123
 - Click Login.

Output:

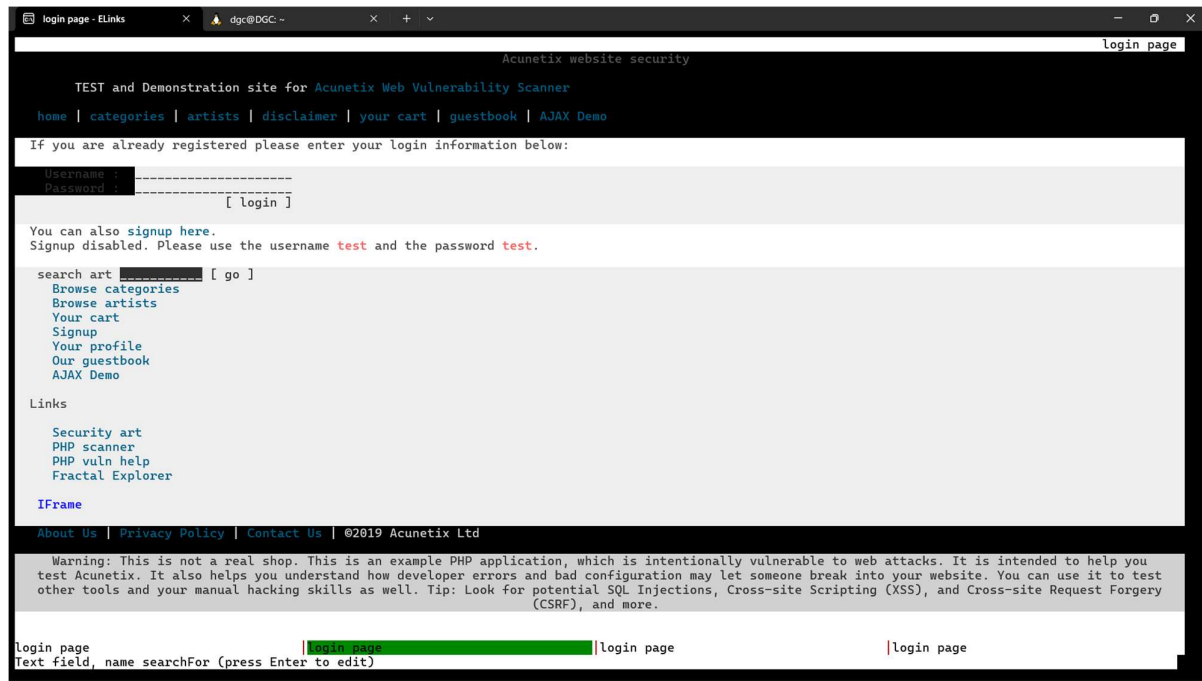


Figure -3.1- result

POST /login.php HTTP/1.1

Host: testphp.vulnweb.com

uname=test&pass=test

Mitigation Steps

Use HTTPS:

Encrypt data in transit to prevent interception.

Implement Strong Authentication:

Use multi-factor authentication (MFA) for added security.

Rate Limiting:

Limit login attempts to prevent brute-force attacks.

Input Validation:

Sanitize and validate user inputs to prevent injection attacks.

Secure Password Storage:

Hash and salt passwords before storing them in the database.

Session Management:

Use secure cookies and implement session timeouts.

Monitor and Log Access:

Keep logs of login attempts and monitor for suspicious activity.

Educate Users:

Inform users about secure password practices and phishing threats.

Regular Security Audits:

Conduct periodic security assessments and vulnerability scans.

Use Web Application Firewalls (WAF):

Protect against common web vulnerabilities and attacks.

 References and Resources Used

1. Wireshark Documentation Packet analysis techniques wiki.wireshark.org
2. OWASP Web Security Testing Ethical hacking guidelines owasp.org
3. GDPR Data protection requirements gdpr-info.eu
4. CVE Details Look up known vulnerabilities cvedetails.com
5. SecLists Common directories/files wordlists github.com/danielmiessler/SecLists
6. Wireshark Network traffic analysis wireshark.org
7. cURL HTTP request generation curl.se